Name	Likith Kumar
Email ID	likithkumarg425@gmail.com
Course Code	102505
Assignment Topic	Automated Enterprise Product Deployment Platform
Source Code Provided	YES
GIT URL	ttps://github.com/LikithKumar0112/Automated-Enterprise-Deployment-Platform-capstone2

Private & Confidential – Do not duplicate or distribute without written permission"

Identified Mistakes and Areas for Improvement Automated Enterprise Product Deployment Platform Review

Problem Statement

The objective of this capstone project is to design an enterprise-grade deployment platform for an analytics application running across multiple environments such as Dev, Staging, and Production.

The project is intended to integrate Jenkins, Terraform, AWS EKS, Docker, Kubernetes, security scanning, monitoring, logging, automated testing, and deployment validation into a single CI/CD workflow.

The expected outcome is an automated deployment process covering infrastructure provisioning, application containerization, Kubernetes deployment, security validation, monitoring, health checks, and production rollback.

Key Observations

1. CI/CD Pipeline Architecture and Jenkins Implementation Issue Identified

The project requires a complete Jenkins pipeline capable of handling application build, testing, security scanning, Docker image creation, infrastructure provisioning, Kubernetes deployment, and post-deployment validation.

Observed Behavior

The submitted Jenkinsfile defines stages for:

- Initialization
- Security scanning
- Build and test
- Terraform planning
- Production approval
- Terraform apply

Private & Confidential – Do not duplicate or distribute without written permission"

- Kubernetes deployment
- Smoke testing
- Monitoring setup

It also includes parameters for environment, Terraform action, testing, security scanning, and image tagging.

The pipeline additionally configures:

- Build timeout
- Retry mechanism
- Build history retention
- Concurrent build prevention
- Kubernetes-based Jenkins agents

Opportunity for Enhancement

The overall pipeline structure is comprehensive, but the submitted package does not contain the repositories and supporting directories referenced by the Jenkinsfile.
For example, the pipeline expects:

- infra/terraform/environments/...
- infra/kubernetes/overlays/...
- infra/monitoring/...
- application Maven source
- EFK configuration

These are not present in the submitted ZIP.

Why This Matters

A CI/CD pipeline must be executable against the submitted project structure. Missing dependencies and directories can prevent the pipeline from progressing beyond the initial stages.

Key Findings

- Multi-stage Jenkins pipeline is designed.
- Dev/Staging/Prod deployment parameters are included.
- Production approval is implemented.
- Retry, timeout, and concurrent-build controls are present.
- Kubernetes-based Jenkins agents are planned.

Private & Confidential – Do not duplicate or distribute without written permission"

- Several pipeline dependencies are missing from the submission.

Suggested Improvements

- Include all repositories or provide the required repository configuration.
- Ensure every directory referenced by the Jenkinsfile exists.
- Validate the complete Jenkins pipeline from checkout through deployment.
- Provide Jenkins build evidence for successful and failed executions.

2. Terraform and AWS EKS Infrastructure

Issue Identified

The infrastructure should be reproducible through Terraform and support an enterprise Kubernetes platform on AWS EKS.

Observed Behavior

The submitted EKS module includes:

- EKS cluster creation
- Private subnet configuration
- Public/private endpoint controls
- EKS secret encryption using KMS
- EKS control-plane logging
- Managed node groups
- Node-group scaling configuration
- Capacity type configuration
- Node labels and taints
- Rolling node updates
- Cluster and node-group outputs

The module also defines AWS, Kubernetes, and Helm provider requirements.

Opportunity for Enhancement

The submitted ZIP contains only the EKS module and does not contain the environment-level Terraform configuration

Private & Confidential – Do not duplicate or distribute without written permission"

referenced by the README and Jenkins pipeline.

The module also references resources and variables such as IAM roles, KMS policy data, cluster variables, and node-group variables that are not included in the submitted package.

Why This Matters

Terraform modules cannot be independently deployed unless their required variables, supporting resources, environment configuration, and provider setup are available.

Key Findings

- EKS architecture is considered.
- KMS encryption is configured for Kubernetes Secrets.
- Control-plane logging is enabled.
- Managed node groups support configurable scaling.
- Node labels and taints are included.
- Environment-level Terraform implementation is missing from the submitted package.

Suggested Improvements

- Include the complete Terraform environment structure.
- Add VPC, subnet, IAM, security, and supporting EKS resources.
- Include variables.tf, provider configuration, and required data sources.
- Run terraform init, terraform validate, and terraform plan.
- Provide Terraform execution evidence for Dev, Staging, and Production.

3. Docker Containerization and Image Security

Issue Identified

The application needs production-oriented container images that can be built consistently and deployed to the Kubernetes platform.

Observed Behavior

Two Dockerfiles are provided.

Private & Confidential – Do not duplicate or distribute without written permission"

The API image:

- Uses Java 11
- Installs security updates
- Creates a non-root user
- Sets ownership and permissions
- Uses container-aware JVM settings
- Exposes port 8080

The web application:

- Uses NGINX Alpine
- Creates a non-root user
- Removes the default NGINX configuration
- Copies application files
- Configures a health check
- Exposes port 8080

The Jenkins pipeline also builds and pushes both images to ECR.

Opportunity for Enhancement

The Dockerfiles reference files that are not present in the ZIP, including:

- API JAR under target/
- application-prod.yml
- nginx.conf
- mime.types
- Web dist/ directory

Therefore, the images cannot be independently built from the submitted package.

Why This Matters

Containerization is only complete when the Docker build context contains all required application artifacts and configuration.

Key Findings

- Non-root execution is implemented.
- API and web images use separate Dockerfiles.
- API JVM container settings are included.

Private & Confidential – Do not duplicate or distribute without written permission"

- Web health checking is included.
- ECR image publishing is designed.
- Required build artifacts/configuration are missing.

Suggested Improvements

- Include the application source/build process.
- Include all required NGINX configuration files.
- Add a reproducible frontend build stage rather than depending on an externally generated dist/ directory.
- Consider multi-stage builds where applicable.
- Demonstrate Docker build, container startup, health check, and image scan results.

4. Kubernetes Deployment and Application Reliability

Issue Identified

The application requires highly available Kubernetes workloads with controlled updates, health monitoring, resource management, and service exposure.

Observed Behavior

The API Deployment includes:

- 3 replicas
- RollingUpdate strategy
- maxSurge: 1
- maxUnavailable: 0
- Pod anti-affinity
- Resource requests and limits
- Liveness probe
- Readiness probe
- Startup probe
- Security context
- ConfigMap-based configuration
- Secret-based environment variables
- ClusterIP Service
- AWS ALB Ingress

Private & Confidential – Do not duplicate or distribute without written permission"

Prometheus scrape annotations are also included.

Opportunity for Enhancement

The API deployment is reasonably detailed, but only the API base manifest is included. The README describes a larger Kubernetes structure involving overlays and additional application components that are absent from the submission.

Why This Matters

Production Kubernetes deployment requires all dependent resources to be available together, including configuration, Secrets, Services, Ingress, overlays, and application images.

Key Findings

- High availability is considered through multiple replicas.
- Rolling updates are configured for controlled deployments.
- Health probes are properly separated.
- Resource requests and limits are defined.
- Pod anti-affinity improves workload distribution.
- Kubernetes security context uses a non-root user.
- Complete environment overlays are missing.

Suggested Improvements

- Add complete base and environment-specific overlays.
- Include API ConfigMap and Secret manifests or a secure secret-management mechanism.
- Include the web application Deployment and Service.
- Validate selectors, probes, Ingress, and Services in a live EKS cluster.
- Demonstrate rolling deployment and rollback.

5. Security and DevSecOps Integration

Issue Identified

An enterprise deployment pipeline should prevent vulnerable application, container, and infrastructure changes from reaching production.

Private & Confidential – Do not duplicate or distribute without written permission"

Observed Behavior

The project includes security scanning using:

- SonarQube
- Snyk in the documented architecture
- Trivy
- Semgrep
- Checkov

The deployment script performs:

- Semgrep scanning
- Container vulnerability scanning
- Terraform/IaC scanning

The Jenkins pipeline also includes SonarQube and Trivy integration.

AWS EKS Secret encryption through KMS is also configured.

Opportunity for Enhancement

The security architecture is described broadly, but several security components are not actually included in the submitted ZIP.

The README references Vault/AWS Secrets Manager, while the Jenkinsfile uses Vault. However, the required Vault integration configuration is not provided.

There is also no actual scan output demonstrating that vulnerabilities are detected and deployment is blocked when thresholds are exceeded.

Why This Matters

Security tooling is effective only when its results influence the deployment decision.

Key Findings

- Multiple security scanning layers are planned.
- SonarQube is included in Jenkins.
- Trivy is used for container scanning.

Private & Confidential – Do not duplicate or distribute without written permission"

- Semgrep and Checkov are included in the deployment script.
- EKS Secrets encryption uses KMS.
- Actual security scan results are not included.
- Vault configuration is not included.

Suggested Improvements

- Provide successful and failed security scan examples.
- Demonstrate a HIGH/CRITICAL vulnerability blocking deployment.
- Include the required Vault integration configuration or use AWS Secrets Manager consistently.
- Avoid static credentials in deployment workflows.
- Document the security gates and their failure conditions.

6. Monitoring, Logging, and Observability

Issue Identified

An enterprise deployment platform needs application and infrastructure visibility after deployment.

Observed Behavior

The project proposes:

- Prometheus
- Grafana
- Alertmanager
- EFK stack
- CloudWatch
- Custom dashboards
- Kubernetes application metrics

The production monitoring values configure:

- Prometheus retention of 30 days
- 100 GiB Prometheus persistent storage
- Grafana persistence
- Grafana dashboard provisioning
- Alertmanager retention
- CPU/memory resource limits

Private & Confidential – Do not duplicate or distribute without written permission"

- Alerts for error rate, crash loops, unavailable replicas, memory usage, and node disk pressure.

Opportunity for Enhancement

The monitoring configuration is one of the stronger parts of the submission, but it is not connected to a complete monitoring deployment within the ZIP.

The Jenkinsfile expects:

infra/monitoring/

and an EFK configuration, but those files are not included.

Why This Matters

Monitoring configuration needs to be deployable and validated against the actual workloads. Otherwise, alert expressions and dashboards cannot be confirmed.

Key Findings

- Prometheus persistence is configured.
- Grafana persistence is configured.
- Alertmanager routing is included.
- Several operational alerts are defined.
- EFK is part of the documented architecture.
- Monitoring deployment files referenced by the pipeline are missing.

Suggested Improvements

- Include the complete monitoring directory.
- Deploy Prometheus/Grafana/Alertmanager in EKS.
- Validate each alert rule against real metrics.
- Include Grafana dashboard screenshots.
- Demonstrate an alert firing and reaching the configured notification channel.
- Include EFK manifests and validate centralized log collection.

7. Deployment Automation, Rollback, and Failure Handling

Private & Confidential – Do not duplicate or distribute without written permission"

Issue Identified

Enterprise deployments require controlled releases and recovery mechanisms when infrastructure or application deployment fails.

Observed Behavior

The project includes:

- Terraform plan/apply/destroy
- Production manual approval
- Kubernetes rolling deployment
- Smoke tests
- Deployment notifications
- Jenkins failure handling
- Production rollback using kubectl rollout undo
- Blue/Green deployment mentioned in the README
- RTO of 4 hours
- RPO of 24 hours
- Target rollback time below 15 minutes

The deployment script also uses an error trap to send a failure notification.

Opportunity for Enhancement

The rollback mechanisms are mainly represented through configuration and documentation. The submission does not provide actual failure/recovery evidence.

There is also a concern with the smoke-test implementation: the shell loops continue after repeated failed checks rather than explicitly failing after all retries are exhausted.

Why This Matters

A deployment pipeline should clearly distinguish between a successful health check and a deployment where all health checks failed.

Key Findings

- Production approval is implemented.
- Kubernetes rollback commands are included.

Private & Confidential – Do not duplicate or distribute without written permission"

- Failure notifications are planned.
- RTO/RPO objectives are documented.
- Smoke tests are included.
- Actual disaster-recovery execution is not demonstrated.
- Some failure paths need stronger exit handling.

Suggested Improvements

- Make smoke tests explicitly fail after the final unsuccessful attempt.
- Demonstrate a failed production deployment.
- Demonstrate Kubernetes rollback.
- Validate that rollback restores the previous working version.
- Demonstrate Terraform state recovery.
- Provide evidence for the documented RTO/RPO procedures.

8. Project Completeness, Reproducibility, and Documentation

Issue Identified

The project is presented as a complete enterprise deployment platform, so the submitted repository should contain the components required to reproduce the documented architecture.

Observed Behavior

The README describes a considerably larger project structure containing:

- Terraform environments
- VPC modules
- Security modules
- Monitoring modules
- Kubernetes overlays
- Helm charts
- Jenkins pipeline libraries
- Job DSL
- Documentation
- EFK

Private & Confidential – Do not duplicate or distribute without written permission"

- Application components
- Deployment scripts

However, the submitted ZIP contains only 20 entries and includes a much smaller subset of that structure.

Several README references point to files/directories that are absent from the submission, including:

- STRUCTURE.md
- docs/architecture.md
- docs/runbooks.md
- docs/incident-response.md
- docs/security.md
- docs/cost-optimization.md
- CONTRIBUTING.md
- LICENSE
- Terraform environment directories
- Kubernetes overlays
- EFK configuration
- Jenkins pipeline library
- Job DSL
- application source/build files

Opportunity for Enhancement

The documentation describes an enterprise platform that is broader than the actual submitted implementation.

The final submission should ensure that the documentation and repository contents represent the same implementation.

Why This Matters

A reviewer or another engineer should be able to clone the submitted project and understand exactly what is implemented, what is required externally, and how to reproduce the deployment.

Key Findings

Private & Confidential – Do not duplicate or distribute without written permission"

- The architecture and intended workflow are clearly documented.
- The project demonstrates awareness of enterprise DevOps practices.
- Multiple technologies are integrated conceptually.
- The submitted repository is incomplete compared with the documented architecture.
- Several pipeline references cannot be resolved from the ZIP.
- End-to-end execution cannot be reproduced from the submitted package alone.

Suggested Improvements

- Include all required project directories and configuration files.
- Remove documentation references to components that are not part of the submission, or add those missing components.
- Provide a complete runnable repository structure.
- Add validation results for Terraform, Docker, Jenkins, Kubernetes, and monitoring.
- Include screenshots/logs showing the complete deployment flow.
- Perform a final clean-clone test to confirm that another engineer can reproduce the setup.

Private & Confidential – Do not duplicate or distribute without written permission"